

Reasoning, Evaluation, Systems and Applications

ECE 247B · LECTURE 3

Shreyas Rajesh

Spring 2026

What changed after chat models:
thinking, measuring, serving, and
building with LLMs.

Lecture 2 Recap: What We Covered

01

Scaling

Loss improves predictably with more compute, but compute allocation between parameters and tokens matters.

02

Pretraining

Next-token prediction over broad data builds a capable base model, not yet a reliable assistant.

03

SFT

Instruction demonstrations teach chat format, task following, and the first layer of assistant behavior.

04

Preferences

RLHF and DPO optimize sampled responses toward behavior people or judges prefer.

Lecture 2 moved from scale to assistant behavior. Today starts at the next question: once the model is optimized for better responses, can we also train and run it to spend more useful computation before answering?

Question: If preference optimization teaches a model which answers are better, what teaches it how to search for better answers?

Sources: Kaplan et al., "Scaling Laws for Neural Language Models," 2020; Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022; Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022; Rafailov et al., "Direct Preference Optimization," 2023.

Lecture 2 Recap: The Pipeline

1. Pretrain

Next-token prediction over broad text, code, math, and web data builds a base model.

2. Allocate compute

Scaling laws and Chinchilla-style analysis ask how to split compute between parameters and tokens.

3. SFT

Instruction data teaches chat format, task following, tone, and basic assistant behavior.

4. Preferences

Human or model comparisons say which sampled responses are better.

5. Optimize

RLHF, DPO, filtering, and eval loops move the policy toward useful assistant behavior.

The same transformer skeleton is still underneath. The behavior changes because the data distribution and objective pressure change.

Sources: Kaplan et al., "Scaling Laws for Neural Language Models," 2020; Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022; Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022; Rafailov et al., "Direct Preference Optimization," 2023.

Lecture 2 Recap: The Objective Shift

Base model / SFT

Likelihood of target text

$$\min_{\theta} -\log \pi_{\theta}(y^* | x)$$

The data gives the target tokens. Training asks the model to imitate them.



Preference / RL

Choose better behavior

$$\max_{\theta} \mathbb{E}[R(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}})$$

The model samples outputs, then a reward or preference signal pushes it toward better responses.

This shift matters for reasoning: if we can score final answers, then the model can be trained to produce solution processes that reach those answers more often.

Sources: Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022; Rafailov et al., "Direct Preference Optimization," 2023; Sutton and Barto, "Reinforcement Learning: An Introduction," 2018.

The Handoff: From Better Answers to Better Search

Chat model question

Given a user prompt, which response is most helpful, safe, and preferred?

Reasoning question

Given a hard problem, what computation should the model do before answering?

Systems question

If answers use more tokens, samples, context, and tools, how do we serve them efficiently?

Lecture 2 stopped here

Post-training aligns the model's output distribution with assistant behavior.

Lecture 3 starts here

Reasoning models use post-training and inference-time compute to make harder problems tractable.

Sources: Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022; OpenAI, "Learning to reason with LLMs," 2024; DeepSeek-AI et al., "DeepSeek-R1," 2025/2026.

Part I: Reasoning Models

From Prompted Reasoning to Trained Reasoning

Chain-of-thought prompting

The model is asked to write intermediate steps before the answer.

```
Question
Worked examples with reasoning
New problem
Let's think step by step...
```

Useful, but the behavior depends heavily on prompting and can be brittle.

Reasoning models

The model is post-trained so extra inference tokens are part of the learned problem-solving behavior.

```
Question
Internal scratchpad or sampled solutions
Verification / selection
Final answer
```

The new ingredient is not just longer output. It is training and selection that make longer computation useful.

Sources: Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022; Kojima et al., "Large Language Models are Zero-Shot Reasoners," 2022; OpenAI, "Learning to reason with LLMs," 2024.

The o1 Moment

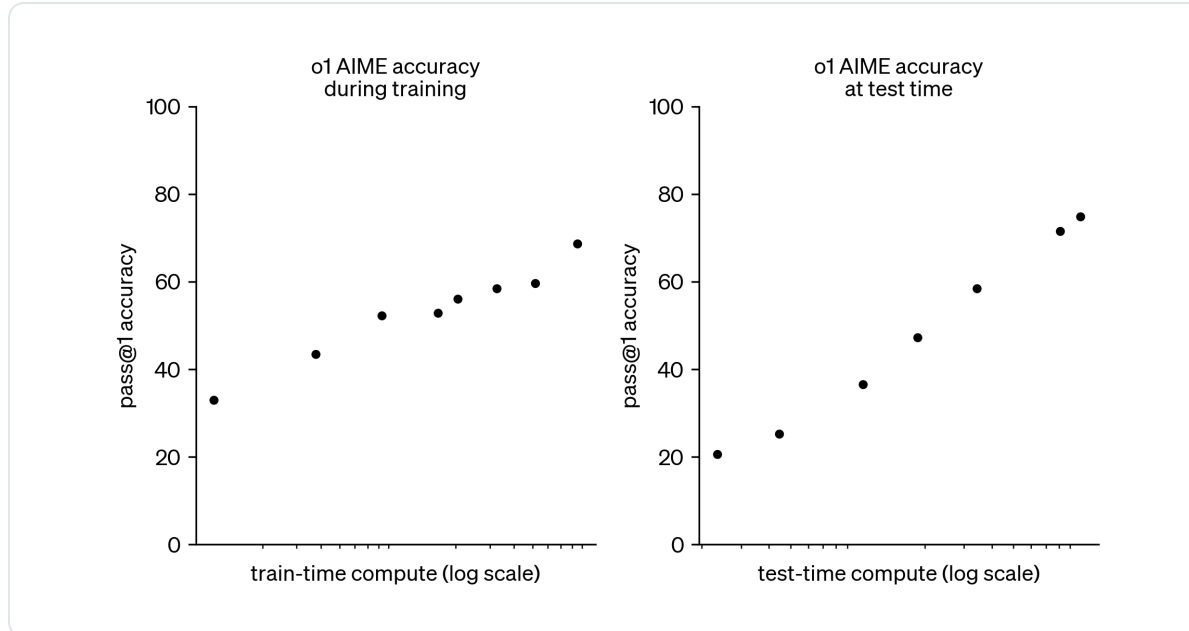


Figure source: OpenAI, "Learning to reason with LLMs," 2024. The plots show AIME pass@1 accuracy versus RL train-time compute and test-time compute.

Source: OpenAI, "Learning to reason with LLMs," 2024.

What became visible

- Reasoning quality improved with more reinforcement-learning training compute.
- It also improved with more test-time compute.
- The public message was a new scaling axis: spend more compute when answering hard questions.
- The exact training and inference recipe was not fully disclosed.

This changed the field's mental model: inference was no longer just "run the model once and sample."

What Makes a Reasoning Model Different?

Training

Reward the outcome

Use tasks where answers can be checked: math answers, code tests, formal proofs, tool-verified work.

Inference

Spend compute adaptively

Let the model generate scratch work, try alternatives, sample multiple solutions, or call verifiers.

Behavior

Learn to check itself

Useful traces often include decomposition, verification, revision, and strategy changes.

Not magic

The model is still autoregressive token prediction under the hood.

New objective pressure

Post-training rewards solution processes that reach correct final answers more often.

Sources: OpenAI, "Learning to reason with LLMs," 2024; DeepSeek-AI et al., "DeepSeek-R1," 2025/2026.

From RLHF to RLVR

Lecture 2

RLHF / DPO

Optimize for responses that humans or judge models prefer.

$$x, y_w, y_l \rightarrow \text{preference signal}$$

- Good for helpfulness, tone, safety, and subjective quality.
- The reward is a learned or implicit proxy.

Today

RLVR

Reinforcement learning with verifiable rewards: optimize for outputs whose final answers can be checked automatically.

$$x, y \rightarrow \text{verify}(x, y) \in \{0, 1\}$$

- Natural for math, code tests, logic puzzles, and tool-checked tasks.
- The reward is sparse, but grounded in correctness.

Question: If only the final answer is rewarded, why would the model learn to produce longer reasoning traces?

Sources: Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022; Rafailov et al., "Direct Preference Optimization," 2023; DeepSeek-AI et al., "DeepSeek-R1," 2025/2026.

RLVR Reward Computation Is Simple

DeepSeek-R1-Zero reward

$$R_{\text{rule}} = R_{\text{acc}} + R_{\text{format}}$$

- **Accuracy:** final answer matches a verifier.
- **Format:** reasoning appears in the required tags / structure.
- No outcome or process neural reward model for R1-Zero reasoning RL.

As code

```
def reward(problem, response):  
    answer = extract_final_answer(response)  
    r_acc = verify(problem, answer)  
    r_fmt = has_required_format(response)  
    return r_acc + r_fmt
```

The reward does not grade every reasoning step. Longer reasoning is useful only when it helps the final answer pass the checker.

Question: Which kinds of tasks have verifiers strong enough for this to work?

Sources: DeepSeek-AI et al., "DeepSeek-R1," 2025/2026; Guo et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," 2025; arXiv source [sn-article.tex](#) and [appendix.tex](#).

What Did DeepSeek Actually Verify?

R1-Zero

Base model + RL directly on reasoning tasks. Rewards were rule-based: answer correctness and required output format.

Main verifiable domains

Math, code competitions, and logical/STEM reasoning where final answers or tests can be checked automatically.

Full R1 adds more

Cold-start data, rejection-sampled reasoning traces, non-reasoning SFT data, and later RL for helpfulness/harmlessness.

```
Released sample pattern
  paraphrased from source files
Prompt
Solve a checkable problem.
```

```
Model output
<think> ... reasoning trace ... </think>
<answer> final answer </answer>
```

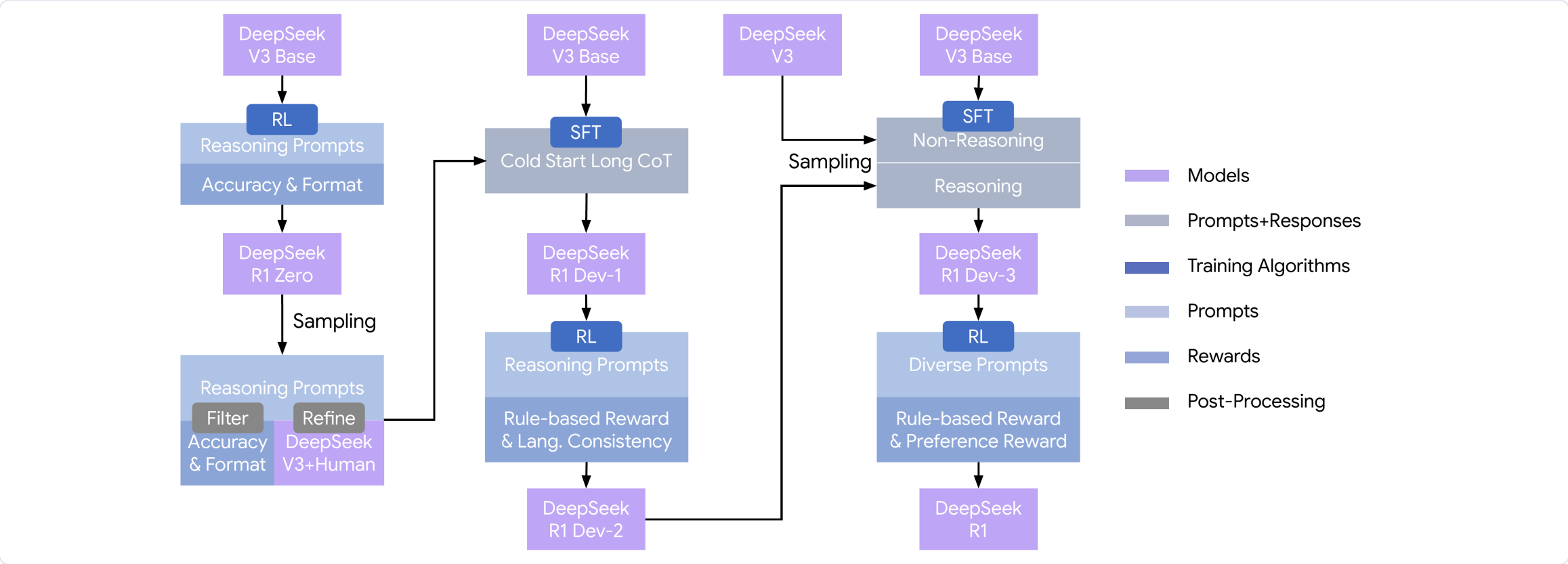
```
Verifier
math: parse boxed/final answer
code: run compiler + tests
```

```
Reward
accuracy reward + format reward
```

Question: If we can score several sampled answers for the same prompt, how should we turn those scores into a policy update?

Sources: DeepSeek-AI et al., "DeepSeek-R1," 2025/2026, arXiv source files `data/sft_sample_math.md`, `data/sft_sample_code.md`, `appendix.tex`; Guo et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," 2025.

DeepSeek-R1: The Paper's Training Pipeline



Read left-to-right: R1-Zero is the pure-RL branch from DeepSeek-V3-Base; full R1 adds cold-start SFT, rule-based reasoning RL, sampling/SFT over reasoning and non-reasoning data, then final RL.

Figure: DeepSeek-AI et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025/2026, arXiv source [figures/R1Pipeline_v0603.pdf](#) , Figure 3.

DeepSeek-R1: A Public Recipe

R1-Zero

Start from a base model and apply RL directly on verifiable reasoning tasks.

Cold start

Add small high-quality reasoning data to improve readability and formatting.

Reasoning RL

Optimize with verifiable rewards and format rewards on math, code, and reasoning-heavy tasks.

Distillation

Use strong reasoning traces to train smaller dense models.

The main pedagogical point: reasoning behavior can be incentivized by RL when correctness is checkable, without requiring humans to label every reasoning step.

Source: DeepSeek-AI et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025/2026.

GRPO: Group-Relative Policy Optimization

PPO-style RLHF

Use a value model

A separate critic estimates the baseline value so rewards can be converted into advantages.

- Stable and general.
- More memory and training machinery.
- Awkward when rewards arrive only after a long answer.

GRPO

Use the group as the baseline

Sample multiple outputs for the same prompt, score them, then normalize each reward relative to the group.

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

q

One prompt

o₁

reward 0

o₂

reward 1

o₃

reward 0

Update

Increase relatively better samples

Sources: Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," 2024; DeepSeek-AI et al., "DeepSeek-R1," 2025/2026; Guo et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," 2025.

DeepSeek-R1-Zero: The “Aha Moment”

DeepSeek-R1-Zero “aha moment” excerpt

The paper reports an intermediate model rethinking its own derivation inside the reasoning trace.

Question

If $a > 1$, solve a nested-radical equation and find the sum of the real solutions.

Response: <think>

Square the equation, isolate the inner radical, expand ...

Check whether the algebraic transformation preserved constraints ...

Wait, wait. Wait. That's an aha moment I can flag here.

Re-evaluate step by step, then continue toward the final answer ...

Source: DeepSeek-AI et al., “DeepSeek-R1,” 2025, arXiv source, Table “aha moment”. Text is abbreviated for slide legibility.

What to notice

- The model revises its own intermediate reasoning instead of only emitting a final answer.
- The trace separates scratch work from the final answer format.
- Extra tokens can be useful when they are used for checking, backtracking, or trying another route.
- This is an example of behavior induced by reward pressure, not a hand-written algorithm.

Figure source: DeepSeek-AI et al., “DeepSeek-R1,” 2025/2026, arXiv source table excerpt, abbreviated for slide legibility.

Source: DeepSeek-AI et al., “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025/2026.

Test-Time Compute: Three Ways to Spend It

Think longer

Generate more intermediate reasoning tokens before committing to the final answer.

Sample more

Generate many candidate solutions and use voting, ranking, or rejection sampling.

Verify more

Run tests, exact answer checkers, learned verifiers, or external tools to select better outputs.

Prompt

Problem statement

Attempt 1

Reasoning path

Attempt 2

Alternate path

Verifier

Check answer or tests

Final

Return selected answer

Sources: Wang et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," 2022; Cobbe et al., "Training Verifiers to Solve Math Word Problems," 2021; OpenAI, "Learning to reason with LLMs," 2024.

Reasoning Models Still Fail

False confidence

A long solution can still be wrong, especially when no verifier exists.

Overthinking

More tokens can add opportunities for drift, contradiction, or spurious checks.

Domain limits

Verifiable RL is easiest where outputs can be automatically graded.

Cost

Reasoning tokens, multiple samples, and verifiers increase latency and serving cost.

Reasoning models shift the frontier, but they do not remove the need for evaluation. They make evaluation more important because behavior depends on inference settings.

Sources: OpenAI, "Learning to reason with LLMs," 2024; DeepSeek-AI et al., "DeepSeek-R1," 2025/2026.

Part II: Evaluation

Why Perplexity Is Not Enough

Pretraining evaluation

Held-out loss asks whether the model assigns high probability to text from the data distribution.

$$\mathcal{L} = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

This is still useful for pretraining and regression checks.

Assistant evaluation

- Did it follow the instruction?
- Was it correct, useful, and calibrated?
- Did it use tools safely?
- Did it solve the task under realistic constraints?

Once a model is an interactive system, evals must measure behavior, not only next-token prediction.

Sources: Liang et al., "Holistic Evaluation of Language Models," 2022; Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023.

Evals Grew With the Models

2018-2019

GLUE and SuperGLUE test broad NLU tasks with fixed datasets and leaderboards.

2020

MMLU measures multitask knowledge and reasoning across many subjects.

2021-2022

HumanEval, GSM8K, MATH, and BIG-bench focus on code, math, and broader capabilities.

2022-2023

HELM, MT-Bench, and Chatbot Arena emphasize scenarios, metrics, and preference.

2023-2025

GPQA, SWE-bench, and HLE target expert knowledge, real codebases, and hard-to-search tasks.

Now

Production teams combine public benchmarks with private regression, safety, and task-specific evals.

Sources: Wang et al., "GLUE," 2018; Wang et al., "SuperGLUE," 2019; Hendrycks et al., "Measuring Massive Multitask Language Understanding," 2020; Srivastava et al., "BIG-bench," 2022; Liang et al., "HELM," 2022; Zheng et al., "MT-Bench and Chatbot Arena," 2023; Jimenez et al., "SWE-bench," 2023; Rein et al., "GPQA," 2023; Phan et al., "Humanity's Last Exam," 2025/2026.

What Do Benchmark Items Look Like?

2020-2021: fixed items, cheap grading

MMLU

Academic multiple choice

Find the degree for the given field extension $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18})$ over $\mathbb{Q}$. choices: 0 / 4 / 2 / 6

Grading: one option label. Good for broad leaderboard comparisons.

GSM8K

Outputs can be checked

Weng earns \$12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?

Grading: final number. The reasoning trace is useful, but the answer is objectively checkable.

from
answering
→
to
doing

2023-2026: expert, hard-to-search, task-like

HLE

Specialist questions

Hummingbirds within Apodiformes ... How many paired tendons are supported by this sesamoid bone?

Grading: exact answer or 5+ choices. The item tests highly specialized knowledge.

SWE-bench

Repair a real codebase

TSQL - L031 incorrectly triggers "Avoid using aliases in join condition" when no join present

Grading: apply a patch to the repo; hidden fail-to-pass tests decide success.

Question: If the benchmark becomes a workflow, what should the eval log besides the final score?

Item excerpts: cais/mmlu; openai/gsm8k; public HLE example; SWE-bench Lite. Sources: Hendrycks et al., "Measuring Massive Multitask Language Understanding," 2020; Cobbe et al., "Training Verifiers to Solve Math Word Problems," 2021; Jimenez et al., "SWE-bench," 2023; Center for AI Safety, Scale AI & HLE Contributors Consortium, "A benchmark of expert-level academic questions to assess AI capabilities," 2026.

Static Capability Benchmarks

Benchmark	What it measures	Why it mattered
MMLU	57 academic and professional subject areas.	A broad test of multitask knowledge and reasoning beyond one NLP task.
BIG-bench	Many diverse tasks contributed by the research community.	Exposed capabilities and failure modes that were missing from older leaderboards.
HELM	Scenarios plus metrics such as accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency.	Shifted evals from one number to a structured measurement framework.
GPQA	Graduate-level biology, physics, and chemistry questions designed to resist simple search.	Hard enough that expert and non-expert performance separate sharply.

Static evals are cheap and repeatable. Their weakness is that they can saturate, leak into training data, or miss interactive behavior.

Sources: Hendrycks et al., "Measuring Massive Multitask Language Understanding," 2020; Srivastava et al., "Beyond the Imitation Game," 2022; Liang et al., "Holistic Evaluation of Language Models," 2022; Rein et al., "GPQA," 2023.

Math and Code Changed the Shape of Evals

HumanEval

Generate Python functions from docstrings and check them with unit tests.

```
def has_close_elements(numbers, threshold):  
    """Check if any two numbers are closer than threshold."""  
    ...
```

GSM8K and verifiers

Grade-school math problems diagnose multi-step reasoning. Candidate solutions can be generated and ranked by verifiers.

generate many → rank/check → return best

Code and math made one thing clear: if the output can be executed or checked, evaluation can become part of inference and training.

Sources: Chen et al., "Evaluating Large Language Models Trained on Code," 2021; Cobbe et al., "Training Verifiers to Solve Math Word Problems," 2021.

Benchmark Snapshot: Open Reasoning Model

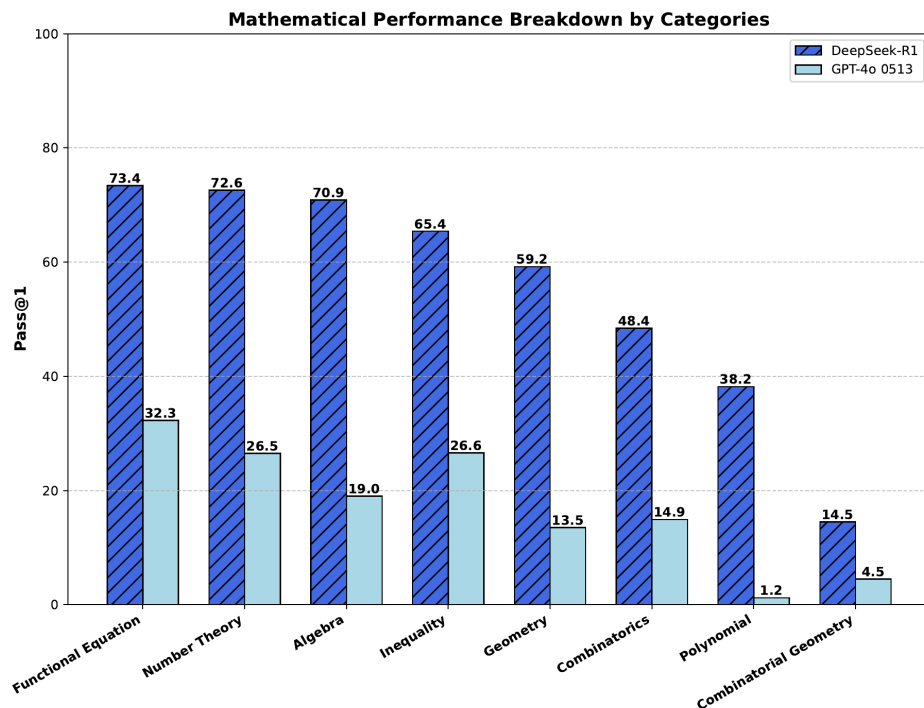


Figure source: DeepSeek-AI et al., "DeepSeek-R1," 2025/2026, arXiv source figure for math benchmark breakdown.

Source: DeepSeek-AI et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025/2026.

How to read this

- Aggregate scores can hide which subskills improved.
- Competition math is a good match for verifiable reasoning RL.
- Pass@1 means one answer attempt, not majority vote over many samples.
- Always ask whether the benchmark matches the capability you care about.

Preference and Judge-Based Evals

Pairwise preference

Show two model responses to the same prompt and ask which is better.

$$y_A \succ y_B \quad \text{or} \quad y_B \succ y_A$$

This matches the RLHF data format and captures open-ended quality.

LLM-as-judge

Use a strong model to judge outputs at scale, then validate judge behavior against human preferences.

- Scalable and fast.
- Can have position, verbosity, and self-preference biases.
- Needs calibration and spot-checking.

Sources: Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022; Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023.

Task Evals: From Answering to Doing

Eval style	Example	What changes
Closed-form QA	MMLU, GPQA, HLE	Model returns an answer; grading is fixed or expert-labeled.
Executable code	HumanEval	Model output is run against tests.
Repository repair	SWE-bench	Model must inspect a codebase, edit files, and satisfy issue tests.
Interactive environment	WebShop, ALFWorld, browser tasks	Model acts over multiple steps and observes state changes.

The frontier moved from "can it answer?" to "can it complete a realistic workflow under constraints?"

Sources: Chen et al., "Evaluating Large Language Models Trained on Code," 2021; Yao et al., "WebShop," 2022; Jimenez et al., "SWE-bench," 2023.

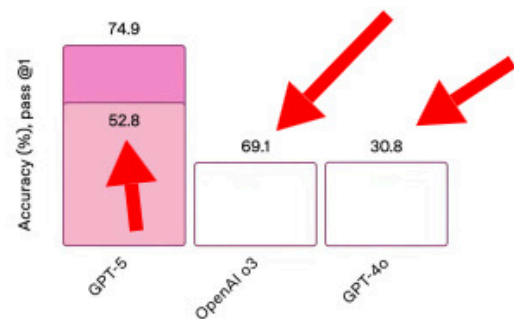
Benchmaxxing: Benchmarks Become Signals

Academic

SWE-bench Verified

Software engineering

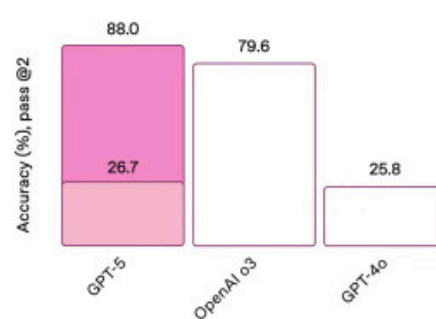
Without thinking With thinking



Aider Polyglot

Multi-language code editing

Without thinking With thinking



Why it happens

Benchmarks are compact public signals: one number can stand in for model quality, product credibility, and research progress.

- They make launches legible.
- They let users compare models quickly.
- They pressure labs to show visible wins.

What to inspect

The chart is not the eval. Ask about the harness, omitted tasks, reasoning effort, tool budget, samples, cost, and uncertainty. Even axis choices and stacked bars can shape the story students take away.

User-provided screenshot excerpt from an OpenAI GPT-5 benchmark presentation; metric values match OpenAI's GPT-5 developer launch page.

Question: What extra metadata should a benchmark chart include before we trust the headline number?

Sources: OpenAI, "Introducing GPT-5 for developers," 2025; OpenAI, "Introducing SWE-bench Verified," 2024; Jimenez et al., "SWE-bench: Can Language Models Resolve Real-world Github Issues?," 2023; Aider, "Aider LLM Leaderboards," 2025/2026.

How Benchmark Tables Mislead

Prompting

Zero-shot, few-shot, CoT, tool access, and formatting can all change scores.

Decoding

Temperature, best-of-N, majority vote, and reasoning effort change the result.

Contamination

Benchmark examples may appear in pretraining, SFT, or benchmark-tuning data.

Saturation

When strong models max out a benchmark, the leaderboard stops separating models.

The fix

Report protocols, not just scores: prompts, decoding, tools, judge, retries, and data filters.

The practical habit

Use public benchmarks for orientation and private task-specific evals for product decisions.

Sources: Liang et al., "Holistic Evaluation of Language Models," 2022; Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023; Sainz et al., "NLP Evaluation in trouble: On the Need to Measure LLM Data Contamination for each Benchmark," 2023.

A Useful Eval Stack

Unit tests

Small deterministic checks for exact behavior.

Task suite

Representative examples from the real use case.

Regression

Keep historical failures from coming back.

Preference

Human or model judgments for open-ended quality.

Safety

Red-team, misuse, privacy, and robustness cases.

Evaluation is not one leaderboard. It is an engineering loop that tells you whether a model change helped the behavior you actually need.

Sources: Liang et al., "Holistic Evaluation of Language Models," 2022; Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," 2022.

Part III: Systems

Why Systems Matter for LLMs

Latency

How long until the first token, and how fast do later tokens stream?

Throughput

How many tokens per second can the serving cluster produce across users?

Memory

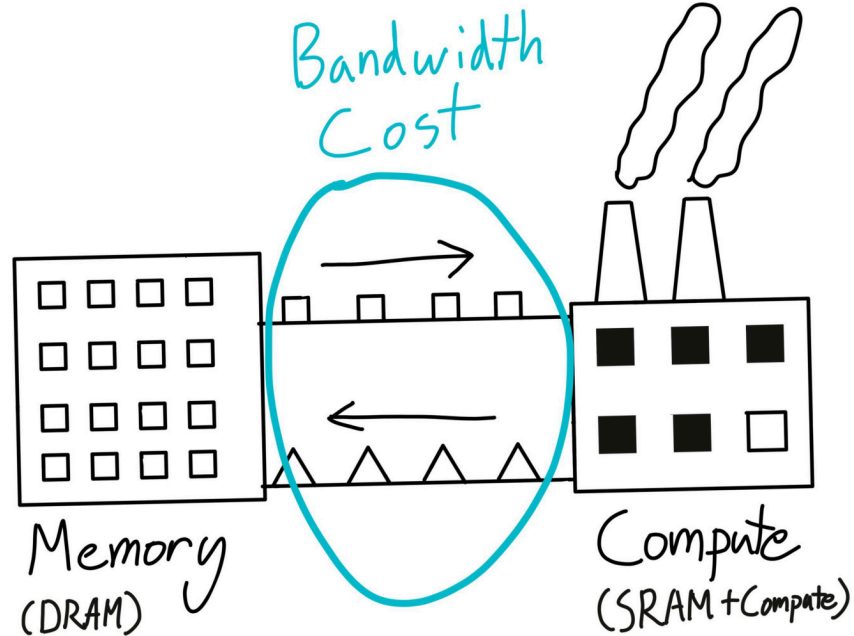
Weights, activations, and KV cache decide which batch sizes and contexts fit.

For LLM inference, the bottleneck is often moving data through memory fast enough, not just doing more floating point operations.

Question: What does "moving data fast enough" mean on a GPU?

Sources: Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," 2019; He, "Making Deep Learning Go Brrrr From First Principles," 2022; NVIDIA Technical Blog, "Mastering LLM Techniques: Inference Optimization," 2023.

GPU Mental Model: Workers Need a Supply Line



DRAM is the warehouse

Weights, activations, and KV cache live in large off-chip memory.

SRAM + tensor cores are the factory

On-chip storage is fast, but small. Tensor cores are useful only when fed.

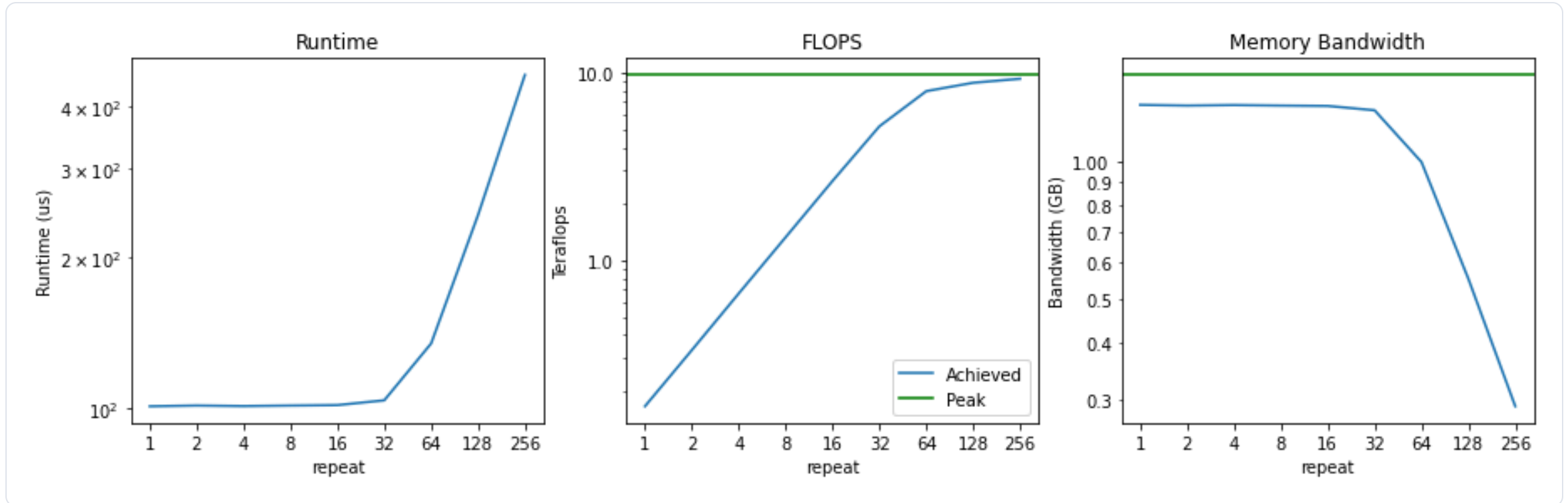
Bandwidth is the supply line

If the supply line is saturated, more peak FLOPs do not help the next token arrive sooner.

Question: If tensor cores are extremely fast, when do they still sit idle?

Figure: He, "Making Deep Learning Go Brrrr From First Principles," 2022. Hardware context: NVIDIA, "NVIDIA A100 Tensor Core GPU," 2020.

Compute-Bound vs. Memory-Bound

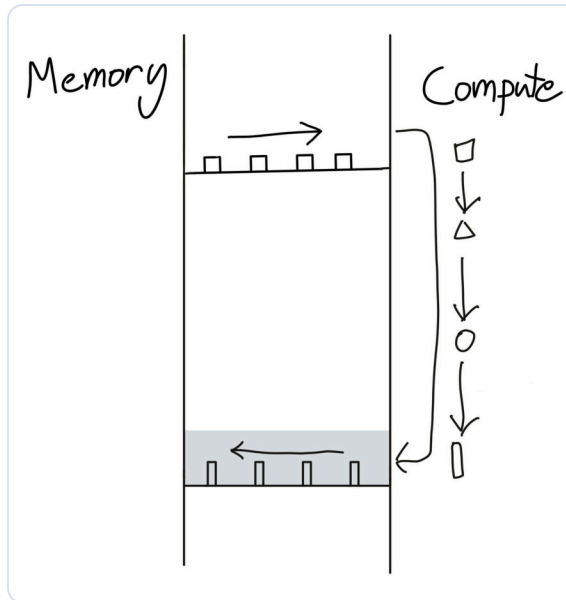
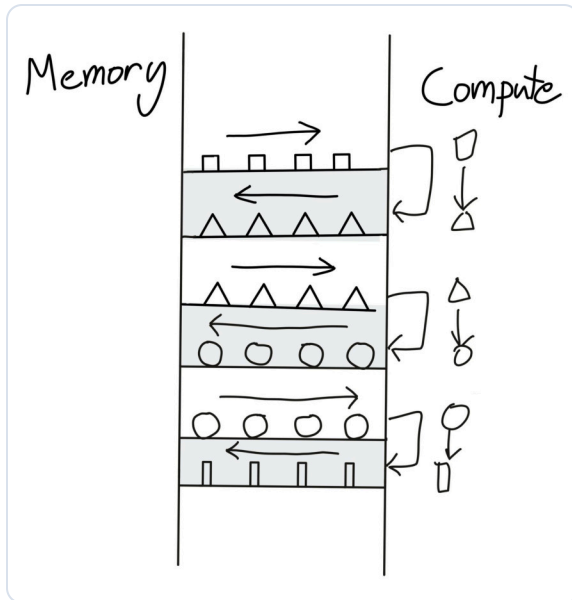


At low arithmetic intensity, memory bandwidth is saturated and achieved FLOPs are low. After enough repeated math per memory read, the workload shifts toward compute-bound behavior.

Question: Which part of LLM inference looks like a big batched matrix multiply, and which part looks like repeated one-token reads?

Figure: He, "Making Deep Learning Go Brrrr From First Principles," 2022. Sources: Williams et al., "Roofline," 2009; NVIDIA Technical Blog, "Mastering LLM Techniques: Inference Optimization," 2023.

Operator Fusion: Do More Before Writing Back



Why fusion helps memory-bound work

Separate pointwise kernels repeatedly read from DRAM, do tiny work, then write intermediate tensors back to DRAM.

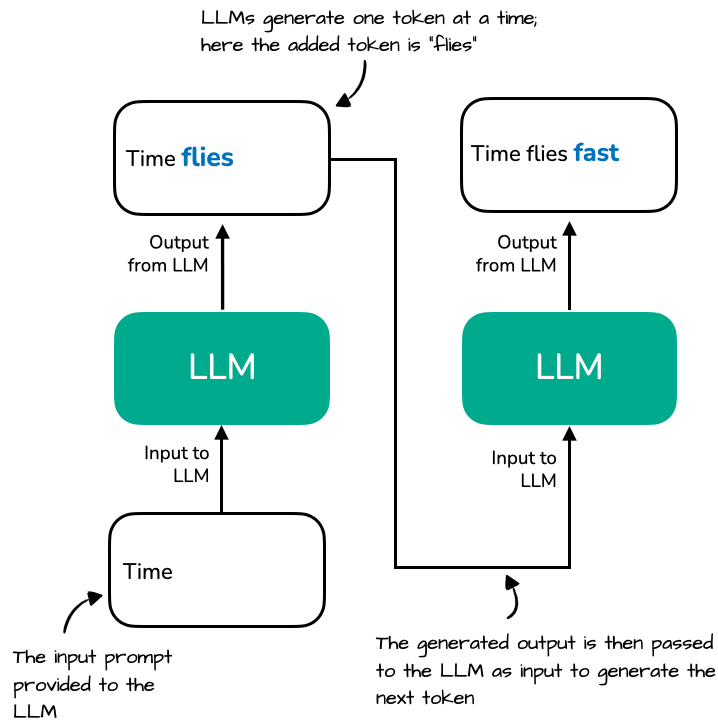
```
x1 = x.cos()      # read x, write x1
x2 = x1.cos()     # read x1, write x2
x2 = fused_cos_cos(x)  # read x, write x2
```

- Fusion removes intermediate global-memory traffic.
- Arithmetic increases, but memory movement can stay nearly fixed.
- This is the compiler version of "keep the data in the factory."

Question: If fusion helps by avoiding extra memory trips, what happens when generation forces us back into a one-token-at-a-time loop?

Figures/source: He, "Making Deep Learning Go Brrrr From First Principles," 2022. Related compiler context: Tillet et al., "Triton: an intermediate language and compiler for tiled neural network computations," 2019.

Autoregressive Inference: One Token at a Time



The dependency

At generation step t , the model samples the next token from the prefix that already exists.

$$p(x_1, \dots, x_T) = \prod_t p(x_t | x_{<t})$$

The next step cannot start until the previous token exists.

The systems consequence

Prompt tokens can be processed together, but generated tokens arrive through a sequential decode loop.

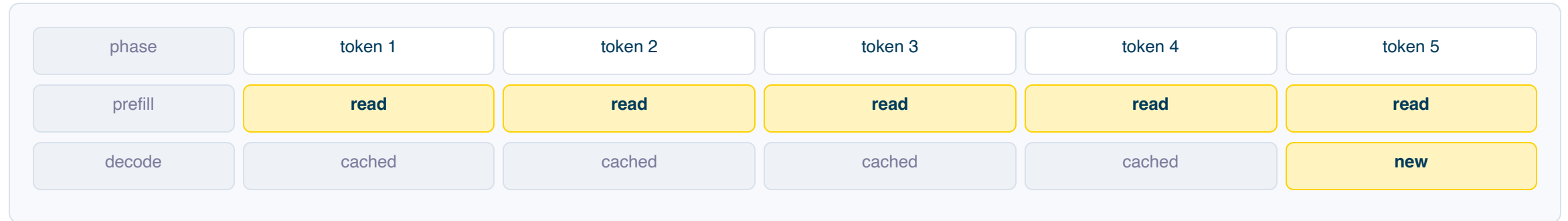
This is why latency, batching, KV cache memory, and per-token bandwidth matter so much in serving.

Figure: Sebastian Raschka, "Understanding and Coding the KV Cache in LLMs from Scratch," 2025.

Question: If the prompt can be processed in parallel but the answer is generated one token at a time, should we treat those two phases the same?

Sources: Bengio et al., "A Neural Probabilistic Language Model," 2003; Radford et al., "Improving Language Understanding by Generative Pre-Training," 2018; Raschka, "Understanding and Coding the KV Cache in LLMs from Scratch," 2025.

LLM Inference Workflow: Prefill Then Decode



Prefill

Process the whole prompt in parallel. This resembles a large matrix-matrix workload and can saturate GPU compute.

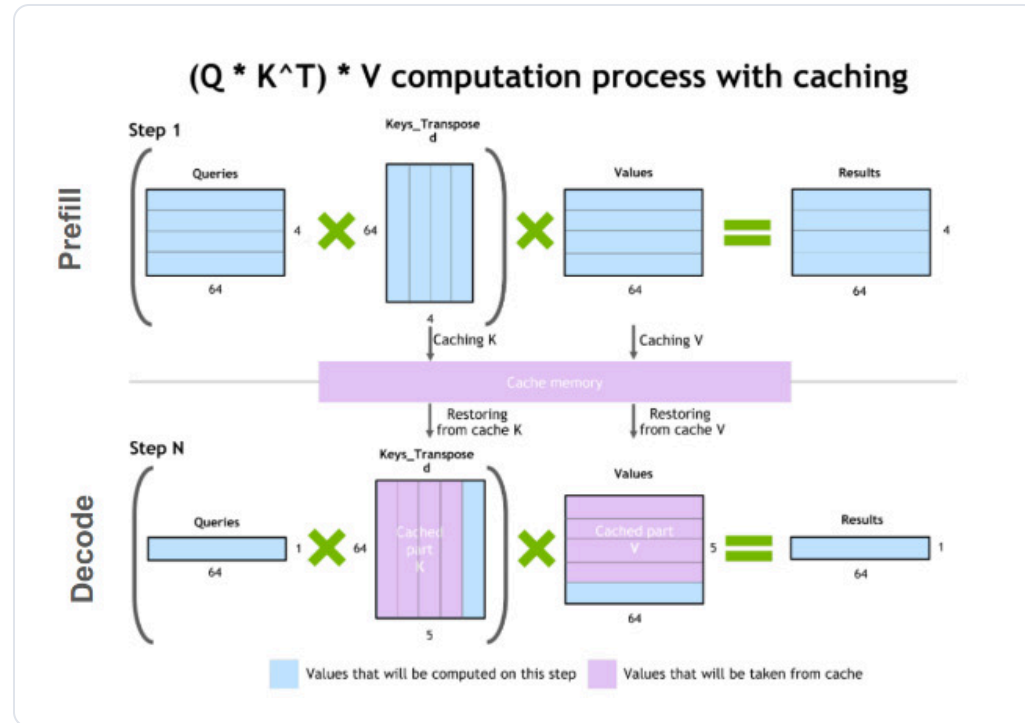
Decode

Generate one new token at a time. This repeatedly reads prior context and is often memory-bound.

Question: Decode needs the previous tokens at every step. What can we save so we do not recompute the whole prefix?

Sources: Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," 2019; NVIDIA Technical Blog, "Mastering LLM Techniques: Inference Optimization," 2023; Kwon et al., "PagedAttention," 2023.

KV Cache: The Basic Trick



Without a cache

Each decode step would rebuild keys and values for the prefix before attending to it.

With a cache

Store prior keys and values in GPU memory. Each step computes only the new token's K/V and appends them.

The trade

Less recomputation, more persistent memory traffic and storage.

Question: If the cache is now the main object we read and store, how large can it get?

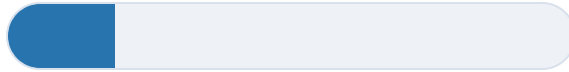
Figure: NVIDIA Technical Blog, Verma, "Mastering LLM Techniques: Inference Optimization," 2023. Sources: Vaswani et al., "Attention Is All You Need," 2017; Shazeer, "Fast Transformer Decoding," 2019.

KV Cache Speedups Are Real, But Conditional

Raschka from-scratch GPT experiment

124M-parameter GPT-style model, 4-token prompt, 200 generated tokens, Mac mini M4 CPU.

No cache



27 tok/s

KV cache



144 tok/s

about 5.3x faster

What changes in real systems?

- The Hugging Face nanoVLM blog reports a 38% generation speedup after adding KV caching.
- The gain grows when repeated prefix recomputation dominates.
- The gain shrinks if cache memory traffic, allocation overhead, batching, or small-model effects dominate.

Takeaway: KV cache removes redundant compute, then memory becomes the thing to optimize.

Question: If KV cache makes decoding faster by storing more state, what resource becomes the next bottleneck?

Sources: Raschka, "Understanding and Coding the KV Cache in LLMs from Scratch," 2025; Hugging Face, Gosthipaty et al., "KV Cache from scratch in nanoVLM," 2025; NVIDIA Technical Blog, Verma, "Mastering LLM Techniques: Inference Optimization," 2023.

KV Cache Size Is a First-Class Constraint

Approximate cache memory

$$2 \cdot L \cdot T \cdot H_{kv} \cdot d_h \cdot \text{bytes}$$

2 for K and V, **L** layers, **T** tokens, **H_{kv}** KV heads, **d_h** head dimension.
Multiply by batch size.

What grows the cache?

- Longer context windows.
- Larger batches.
- More layers and KV heads.
- Higher precision cache storage.

Question: Can we keep many query heads, but store fewer key/value heads?

Sources: Shazeer, "Fast Transformer Decoding," 2019; NVIDIA Technical Blog, Verma, "Mastering LLM Techniques: Inference Optimization," 2023; Ainslie et al., "GQA," 2023; Kwon et al., "PagedAttention," 2023.

MHA, MQA, and GQA

Attention variant	KV heads	Serving implication
Multi-head attention	One K/V head per query head.	Highest KV-cache memory and bandwidth.
Multi-query attention	One shared K/V head for all query heads.	Much smaller cache, but can hurt quality.
Grouped-query attention	Several K/V heads shared by groups of query heads.	Middle ground: much cheaper than MHA, often better quality than MQA.

Idea: GQA reduces how much KV data every decode step must read. **Question:** Can we also reduce memory traffic inside attention itself?

Sources: Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," 2019; Ainslie et al., "GQA," 2023.

FlashAttention: Same Attention, Better Memory Traffic

Standard attention problem

The attention matrix is large, and moving it between GPU memory levels is expensive.

$$\text{softmax}(QK^{\top}/\sqrt{d})V$$

FlashAttention idea

Tile the computation so blocks of Q, K, and V stay in faster on-chip memory, while computing exact attention.

- Reduces reads and writes to high-bandwidth memory.
- Improves wall-clock speed.
- Enables longer contexts in practice.

Idea: The math is unchanged; the implementation respects the memory hierarchy. **Question:** Once the cache exists, how do we manage cache memory across many live requests?

Source: Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022.

Batching and PagedAttention

The serving problem

- Requests arrive with different prompt lengths.
- Generation lengths are not known ahead of time.
- KV cache memory grows and shrinks dynamically.
- Fragmentation lowers batch size and throughput.

PagedAttention

Manage KV cache like virtual memory: split it into blocks, allocate them non-contiguously, and map logical token positions to physical cache blocks.

block

block

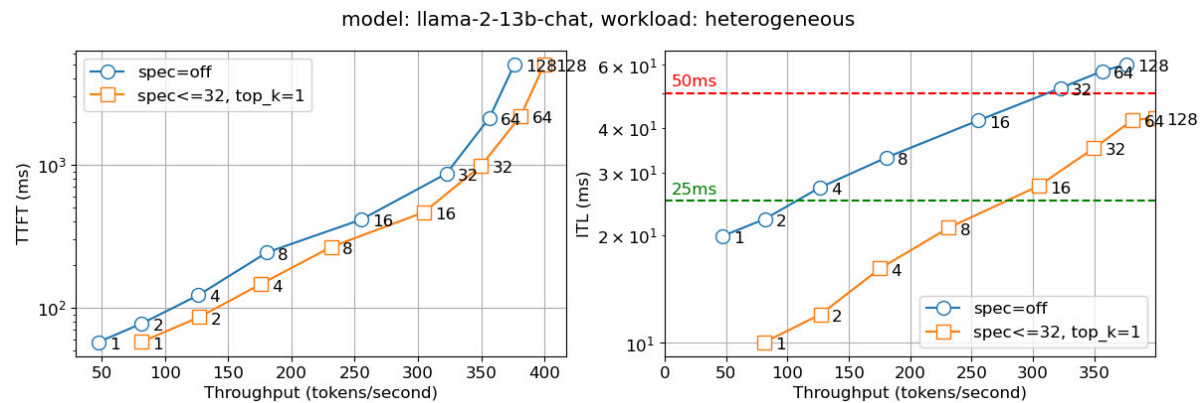
block

shared

Question: We have optimized inference memory and attention traffic. Can we reduce how many full target-model passes we need during decoding?

Source: Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," 2023.

Speculative Decoding: Speedup Evidence



Core idea

A cheap draft mechanism proposes several future tokens. The large target model verifies them in parallel and accepts the longest valid prefix.

~2x

language-model speedup reported in
IBM/PyTorch deployment

~3x

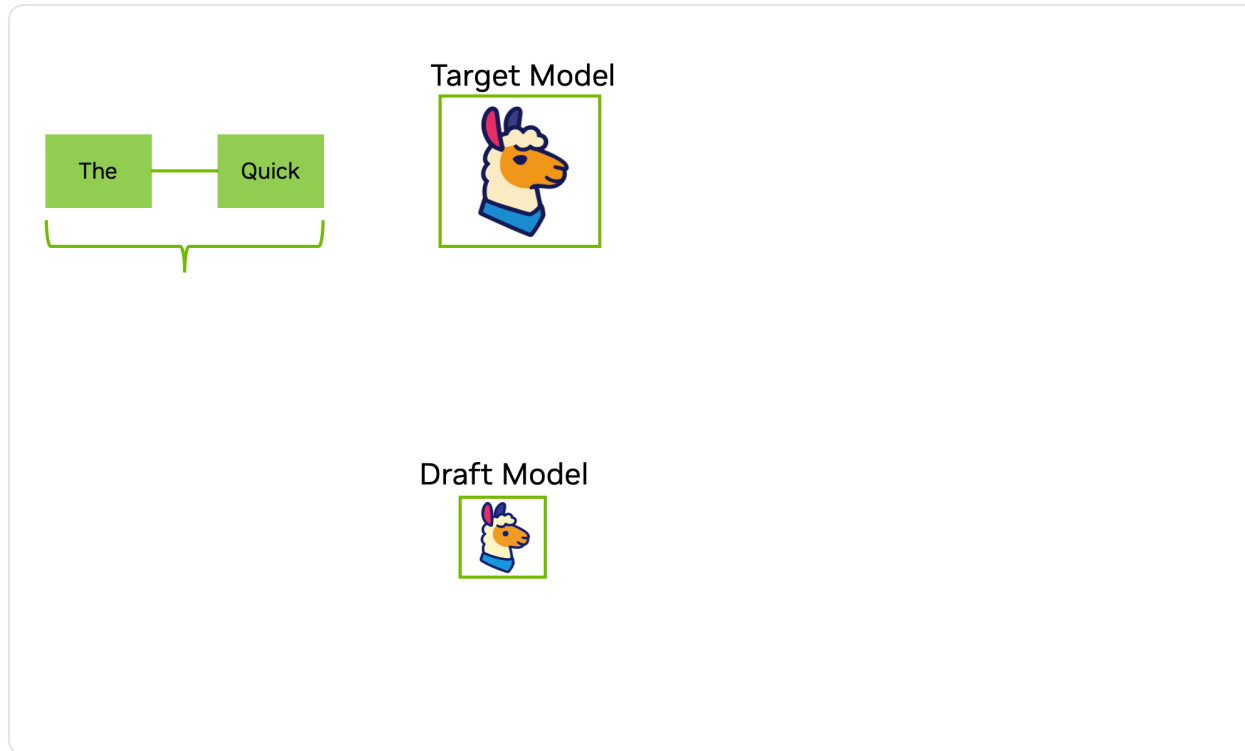
Granite 20B code-model speedup reported in
the same blog

- Best when the draft is accurate and target-model memory traffic dominates.
- Worst case: bad drafts waste extra compute and reduce the gain.

Question: How can a faster approximate draft preserve the exact target model's output distribution?

Figure/source: Team PyTorch at IBM, "A Hitchhiker's Guide to Speculative Decoding," 2024. Sources: Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," 2023; Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," 2023.

Speculative Decoding: Draft Then Verify



Draft-target implementation

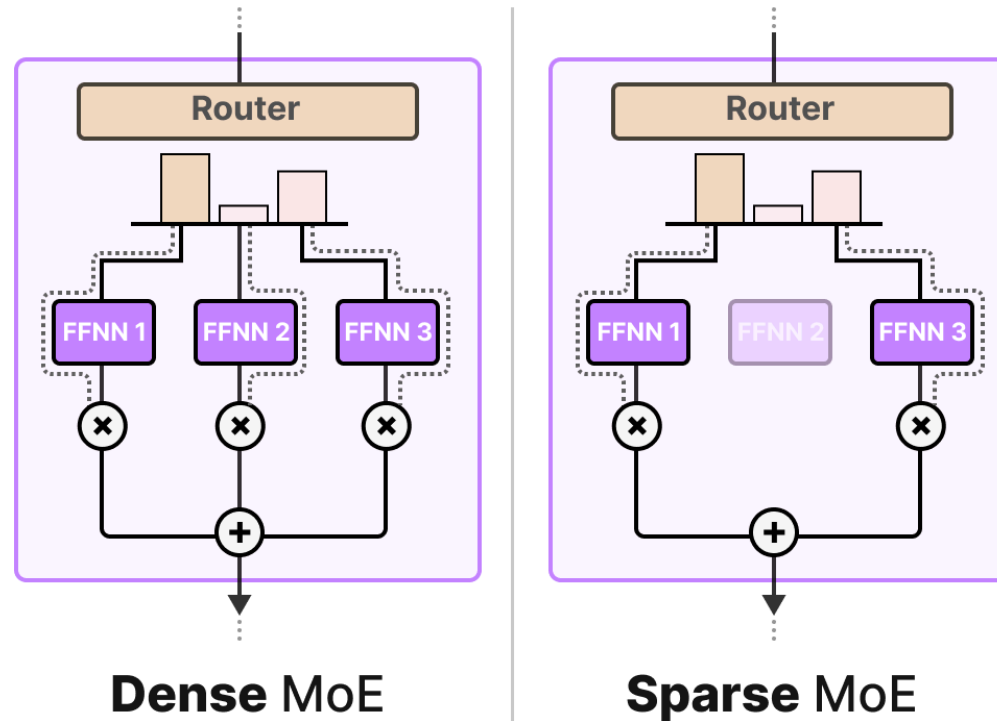
- **Draft:** a smaller model or prediction head proposes several candidate tokens.
- **Verify:** the target model scores the prefix plus all drafted tokens in one forward pass.
- **Accept/reject:** keep the longest prefix that matches the target distribution, then discard the rest.
- **Continue:** update the KV cache and repeat from the accepted prefix.

Speculation trades a little extra compute for fewer sequential target-model round trips.

Question: That reduces sequential decoding work. Can we also increase model capacity without running every parameter for every token?

Figure/source: Li, Yu, and Guo, NVIDIA Technical Blog, "An Introduction to Speculative Decoding for Reducing Latency in AI Inference," 2025. Sources: Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," 2023; Li et al., "EAGLE," 2024.

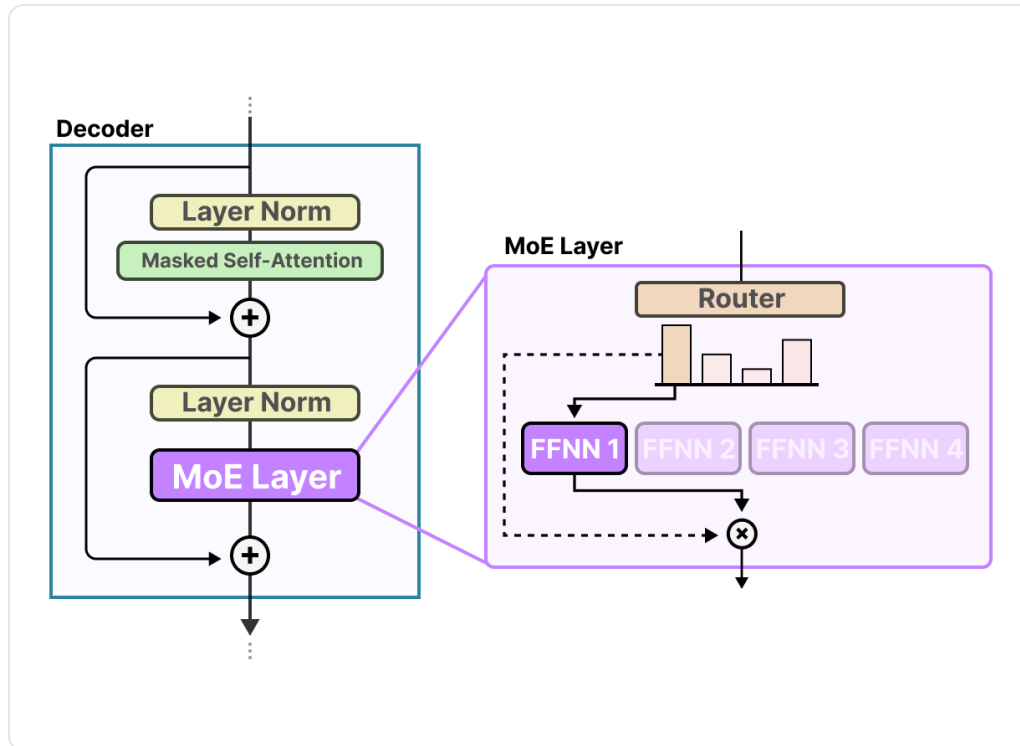
MoE: More Parameters Than Compute Per Token



Idea: MoE decouples parameter count from per-token compute. **Question:** If only selected experts run, where exactly does this layer sit inside a transformer block?

Figure: Maarten Grootendorst, "A Visual Guide to Mixture of Experts (MoE)," 2024. Sources: Shazeer et al., "Outrageously Large Neural Networks," 2017; Fedus et al., "Switch Transformers," 2021; Jiang et al., "Mixtral of Experts," 2024.

MoE: Where It Lives in the Block



Dense transformer layer

Attention mixes information across tokens; the feed-forward network then transforms each token independently.

MoE transformer layer

Replace one dense FFN with many FFNs. A router chooses a small subset of experts for each token.

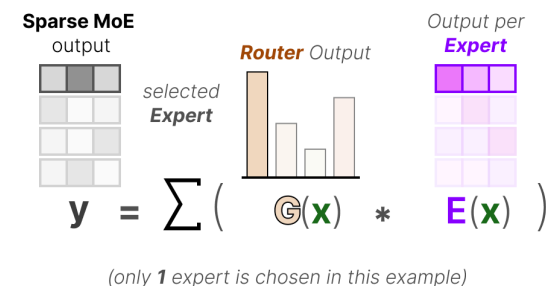
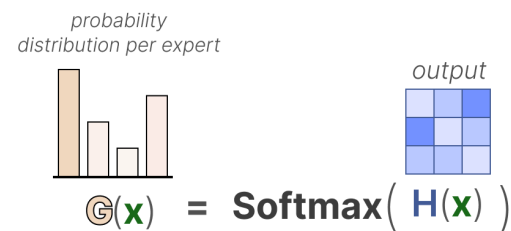
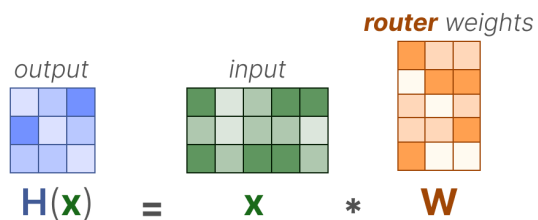
Key trade

Many parameters are stored, but only selected experts run for a given token.

Question: The router chooses a subset for each token. What computation turns a hidden state into an expert choice?

Figure: Maarten Grootendorst, "A Visual Guide to Mixture of Experts (MoE)," 2024. Sources: Shazeer et al., "Outrageously Large Neural Networks," 2017; Fedus et al., "Switch Transformers," 2021.

Router: How a Token Chooses Experts



Per-token routing

The hidden state h is projected to expert logits, normalized into weights, then sparsified by selecting a small number of experts.

Combine step

The layer output is a weighted sum of the selected experts' FFN outputs:

$$\mathbf{y} = \sum_{i \in S} g_i(h) E_i(h).$$

Question: Should routing always mean tokens choose experts, or can experts choose which tokens they process?

Figures: Maarten Grootendorst, "A Visual Guide to Mixture of Experts (MoE)," 2024. Sources: Shazeer et al., "Outrageously Large Neural Networks," 2017; Jiang et al., "Mixtral of Experts," 2024.

Routing Choices: Selection and Capacity

Token-choice routing

Each token chooses a fixed number of experts. Switch uses one selected expert; Mixtral uses two selected experts.

Expert-choice routing

Each expert chooses a fixed-size bucket of tokens. This directly controls expert load, but a token may receive variable compute.

Capacity

Experts have token limits per batch. Too little capacity drops or reroutes tokens; too much capacity wastes compute.

Idea: Routing is a modeling choice and a scheduling choice at the same time. **Question:** What stops the router from sending everything to the same few experts?

Sources: Fedus et al., "Switch Transformers," 2021; Zhou et al., "Mixture-of-Experts with Expert Choice Routing," 2022; Jiang et al., "Mixtral of Experts," 2024.

Load Balancing: The Router Can Collapse

Failure mode

If the router sends most tokens to the same few experts, those experts bottleneck the batch and the others learn little.

- Training becomes uneven.
- Serving latency follows the busiest expert.
- Unused capacity becomes wasted memory.

Typical fixes

MoE systems add load-balancing losses, routing noise, capacity factors, and sometimes expert parallelism so work is spread across devices.

$$\text{quality} + \lambda \cdot \text{balance}$$

Question: If the router is balanced, what kinds of patterns might different experts actually learn?

Sources: Shazeer et al., "Outrageously Large Neural Networks," 2017; Fedus et al., "Switch Transformers," 2021; Lepikhin et al., "GShard," 2020.

What Does an Expert Learn?

Not hand-written roles

Experts are learned FFNs. We do not tell one expert to be "math" and another to be "syntax."

Useful intuition

Routing can separate token/context patterns: language, format, punctuation, code-like structure, arithmetic-like contexts, or style.

Important caveat

Specialization is emergent and messy. Experts often capture overlapping statistical niches, not clean human categories.

Interpretation

Inspecting expert assignments can be useful for debugging, but expert labels are post-hoc explanations.

Architecture trend

Some MoE designs add shared experts so common knowledge is always available while routed experts handle specialization.

Question: Even if experts specialize usefully, what does this routed computation do to serving?

Sources: Zoph et al., "ST-MoE," 2022; Dai et al., "DeepSeekMoE," 2024; Jiang et al., "Mixtral of Experts," 2024; Maarten Grootendorst, "A Visual Guide to Mixture of Experts (MoE)," 2024.

MoE Systems View

Memory

All experts' weights must live somewhere, even if each token uses only a subset.

Communication

Tokens may need to move to the device that owns the chosen expert.

Parallelism

Expert parallelism can scale parameters, but stragglers determine latency.

Serving

Batching gets harder because different tokens activate different computation paths.

Idea: MoE buys model capacity by routing compute. **Question:** What other systems trick attacks memory and bandwidth directly?

Sources: Lepikhin et al., "GShard," 2020; Fedus et al., "Switch Transformers," 2021; Du et al., "GLaM," 2021; Jiang et al., "Mixtral of Experts," 2024.

Quantization: Shrink the Model, Usually Carefully

Weights

Store model weights in lower precision, such as INT8, INT4, or FP8-like formats.

Activations

Quantize intermediate values or use mixed precision to reduce bandwidth and memory.

KV cache

Quantize cached keys and values to fit longer contexts or larger batches.

Benefit

Lower memory, cheaper serving, often faster inference.

Risk

Quality loss is uneven: outliers, long-context behavior, and small models can be sensitive.

Sources: Dettmers et al., "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," 2022; Frantar et al., "GPTQ," 2022; Xiao et al., "SmoothQuant," 2022.

The Systems Tradeoff Table

Technique	Main win	Main tradeoff
KV cache	Avoid recomputing past tokens during generation.	Memory grows with context, batch, layers, and KV heads.
GQA/MQA	Reduce KV cache memory and decode bandwidth.	Too few KV heads can hurt quality.
FlashAttention	Reduce attention memory traffic while keeping exact attention.	Kernel complexity and hardware-specific tuning.
PagedAttention	Higher serving throughput through better cache memory management.	Scheduler and block-management complexity.
MoE	More parameters without proportional per-token FLOPs.	Routing, load balancing, communication, and deployment complexity.
Quantization	Lower memory and often faster serving.	Possible quality loss and calibration burden.

Modern LLM architecture is partly an answer to serving pressure: long contexts, low latency, high throughput, and acceptable cost.

Sources: Ainslie et al., "GQA," 2023; Dao et al., "FlashAttention," 2022; Kwon et al., "PagedAttention," 2023; Shazeer et al., "Sparsely-Gated MoE," 2017; Dettmers et al., "LLM.int8()," 2022.

Part IV: Applications

The LLM Application Pattern

User task

A user asks for an answer, a decision, an edit, a plan, or an action.

Context layer

Retrieve documents, load files, inspect database rows, or gather tool outputs.

Model call

Prompt the model with instructions, context, schema, and available tools.

Action layer

Return text, call functions, write code, update systems, or ask for clarification.

Eval loop

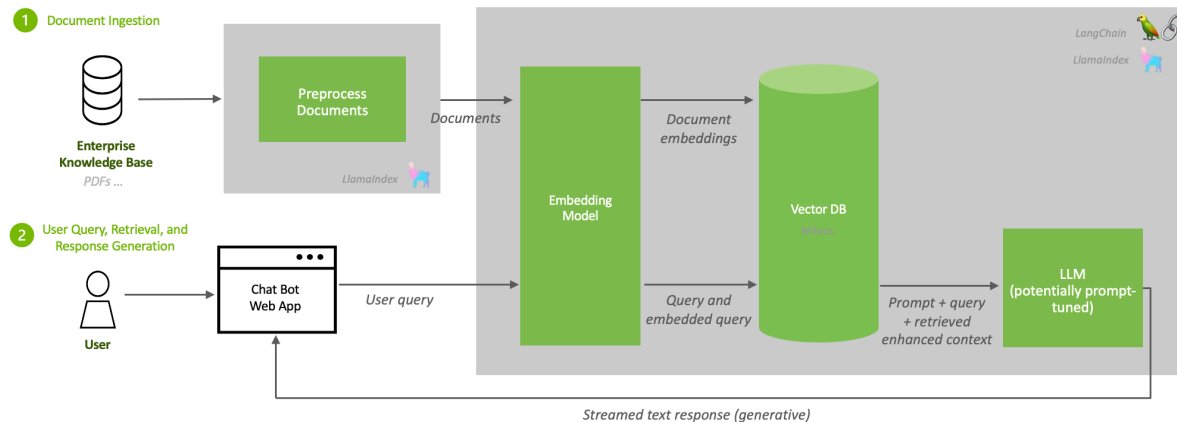
Check correctness, safety, formatting, citations, tests, and user-facing behavior.

In production, the model is usually one component inside a larger data, tool, and evaluation system.

Sources: Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020; Karpas et al., "MRKL Systems," 2022; Yao et al., "ReAct," 2022.

RAG: Retrieval-Augmented Generation

Retrieval Augmented Generation (RAG) Sequence Diagram



Offline ingest

Load documents, split them, embed chunks, and store vectors plus metadata.

Online retrieval

Embed the user query, retrieve relevant chunks, and pack evidence into the prompt.

Grounded generation

The answer is conditioned on retrieved context, then checked for citation support.

Question: If RAG gives the model external memory, where can the system still fail before generation even starts?

Figure: NVIDIA Technical Blog, Wolff, "RAG 101: Demystifying Retrieval-Augmented Generation Pipelines," 2023. Sources: Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020; Guu et al., "REALM," 2020; Izacard and Grave, "Leveraging Passage Retrieval with Generative Models," 2020.

RAG Design Choices

Choice	What can go wrong	Common fix
Chunking	Important facts split across chunks, or chunks are too large to retrieve precisely.	Use semantic chunking, overlap, metadata, and hierarchical retrieval.
Retriever	Embedding search misses lexical or rare terms.	Hybrid retrieval: dense plus BM25, reranking, query rewriting.
Context packing	Too much retrieved text dilutes the prompt.	Rerank, deduplicate, compress, and order by usefulness.
Generation	Model uses prior knowledge instead of retrieved evidence.	Ask for citations, quote spans, or answer only from provided context.
Evaluation	Answer looks plausible but is unsupported.	Faithfulness checks, citation audits, and answerability tests.

Sources: Lewis et al., "Retrieval-Augmented Generation," 2020; Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," 2023.

When to RAG, Fine-Tune, or Tool-Call

Need	Good fit	Reason
Fresh or private facts	RAG	Knowledge can change without changing model weights.
New style or task format	SFT / LoRA	Fine-tuning changes behavior and formatting patterns.
Exact arithmetic or current state	Tool calling	External systems are more reliable for calculation, lookup, and actions.
Multi-step workflow	Agent loop	The model must observe, decide, act, and recover from intermediate failures.

RAG is not a universal replacement for fine-tuning. It solves a different problem: grounding generation in retrieved external context.

Sources: Lewis et al., "Retrieval-Augmented Generation," 2020; Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021; Schick et al., "Toolformer," 2023.

Tool Calling

Language model alone

Good at interpreting intent and producing text, but weak for exact current state.

```
User: What meetings do I have after lunch?  
Model: likely answer from memory? Not enough.
```

Model plus tools

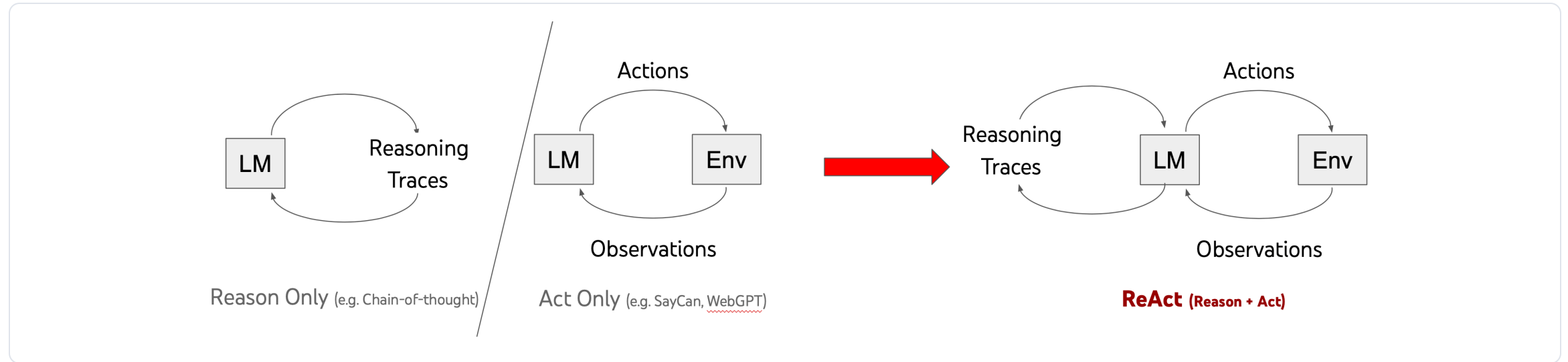
The model emits a structured call; the system executes it and returns an observation.

```
call calendar.search({  
  "date": "2026-05-06",  
  "after": "12:00"  
})
```

Tool calling turns the model into a controller over external APIs. The hard part is making tool choice, arguments, permissions, and error handling reliable.

Sources: Karpas et al., "MRKL Systems," 2022; Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," 2023.

ReAct: Reason and Act Interleaved



Reason only

Can decompose the problem, but may hallucinate facts because it never checks the world.

Act only

Can gather observations, but may fail to synthesize them into a coherent plan.

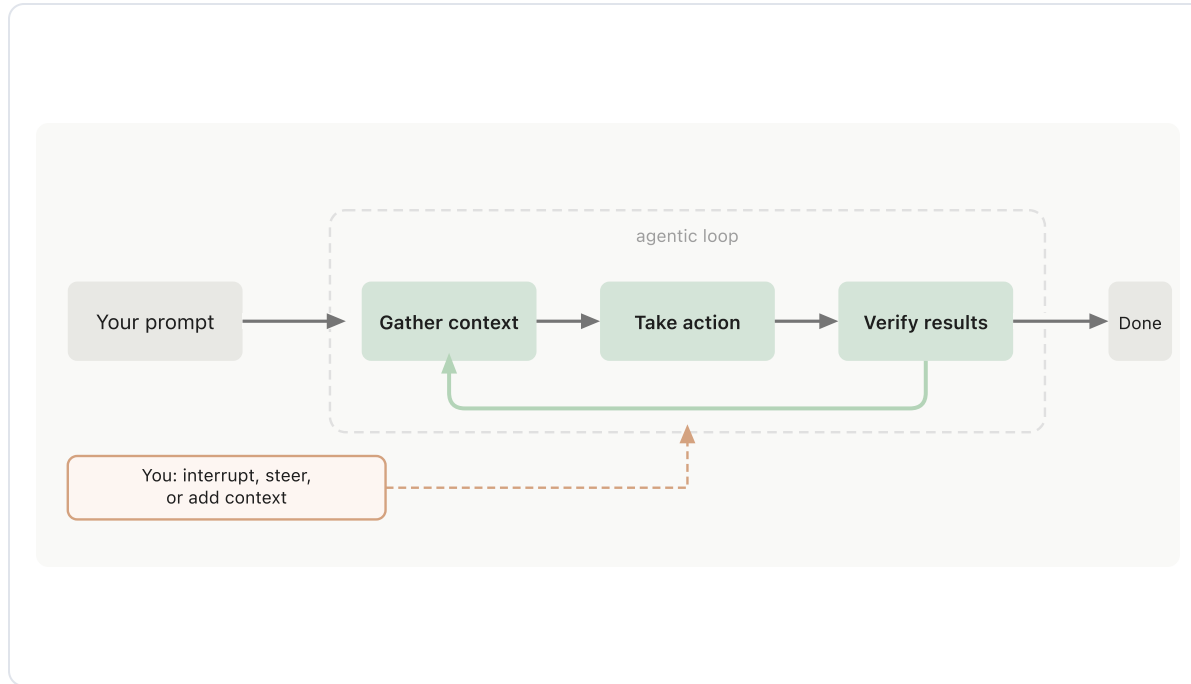
ReAct

Interleave reasoning traces, actions, and observations so each step can update the next.

Question: What changes when the "environment" is not Wikipedia, but a real codebase or computer?

Figure/source: Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," 2022; project page: react-lm.github.io.

Agent Loop in Claude Code



The model is inside a harness

- It gathers context from files, search, command output, web docs, and project instructions.
- It takes actions through tools: read, edit, run commands, use git, call MCP services.
- It verifies with tests, build output, type checks, diffs, and user steering.

context

tools

verification

Question: If tool results change the next model call, why is an agent more like a control loop than a single prompt?

Figure/source: Anthropic, "How Claude Code works," accessed May 6, 2026; Anthropic, "How the agent loop works," accessed May 6, 2026.

Coding Agents: Reasoning Meets Tool Traces

```
agent session

$ Fix the failing auth test, then run the suite.

tool: Grep
search for failing test name and auth handler

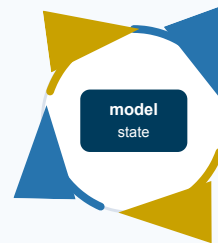
observation
token refresh path skips expired-session check

tool: Edit
patch auth.ts and add regression coverage

tool: Bash
npm test -- auth

final
3 tests pass. Auth regression fixed.
```

What repeats?



1. **Observe** files, errors, docs

2. **Decide** next tool or answer

3. **Act** read, edit, run, call

4. **Verify** tests and checks

What students should notice

- The visible artifact is a sequence of tool calls and observations, not just a final answer.
- Private reasoning is usually summarized or hidden; reliability comes from external checks.
- Long tasks need budgets, permissions, checkpoints, and context management.

Current agent products are useful because reasoning models got better at deciding when to search, edit, run, verify, and stop.

Diagram created for ECE 247B, based on Anthropic, "How Claude Code works," accessed May 6, 2026; Anthropic, "How the agent loop works," accessed May 6, 2026; Yao et al., "ReAct," 2022.

Agents Need Guardrails

State

What has been observed, changed, or committed so far?

Permissions

Which actions are read-only, reversible, or externally visible?

Verification

Which tests, checks, or human approvals are required before acting?

Recovery

How does the agent detect errors, backtrack, or ask for help?

Why this matters

Autonomous loops amplify both capability and failure. A small hallucination can become a real action.

Engineering pattern

Constrain tools, log actions, validate outputs, limit budgets, and require approval for high-impact changes.

Sources: Yao et al., "ReAct," 2022; Jimenez et al., "SWE-bench," 2023; Karpas et al., "MRKL Systems," 2022.

Putting It Together

Reasoning

Inference becomes compute allocation

Hard problems can use more tokens, more samples, and verifiers.

Evaluation

Behavior needs measurement

One public benchmark score is not a product-quality eval.

Systems

Serving shapes architecture

KV cache, GQA, FlashAttention, batching, quantization, and MoE are practical constraints.

Applications

Models sit inside systems

RAG, tools, agents, and eval loops turn a language model into a useful workflow.

The core transformer still matters. Modern LLM practice is what we build around it: post-training objectives, eval protocols, serving systems, retrieval, tools, and action loops.

What You Should Remember

- Reasoning models are post-trained to make extra inference compute useful.
- Verifiable rewards make math, code, and tool-checked tasks especially important.
- Evals evolved from static NLP leaderboards to behavior, preference, expert, and task-completion measurement.
- LLM serving is dominated by memory and scheduling constraints as much as model math.
- RAG, tool calling, and agents are systems patterns, not just prompting tricks.

The practical question is always: what should the model know in weights, what should it retrieve, what should it call, and how will we know it worked?

Synthesis based on sources cited throughout the lecture, especially OpenAI, "Learning to reason with LLMs," 2024; DeepSeek-AI et al., "DeepSeek-R1," 2025/2026; Liang et al., "HELM," 2022; Kwon et al., "PagedAttention," 2023; Lewis et al., "RAG," 2020; Yao et al., "ReAct," 2022.

What We Did Not Cover

Pretraining at scale

Data filtering, deduplication, contamination, curriculum, distributed training, and scaling-law tradeoffs.

Multimodal models

Vision-language models, speech, video, image generation, and how modality adapters connect to LLMs.

Safety and security

Jailbreaks, prompt injection, privacy, red teaming, eval governance, and deployment policy.

Interpretability

Mechanistic circuits, attribution, representation analysis, and why models behave as they do.

Long context

Context extension, retrieval-memory hybrids, positional extrapolation, and context compression.

Training recipes

Optimizer details, parallelism, checkpointing, data mixtures, distillation, and synthetic data loops.

Product evaluation

Human factors, cost-latency tradeoffs, monitoring, A/B tests, and real user workflows.

The open frontier

We are still learning which capabilities come from scale, data, tools, search, and interaction.

This lecture is a map, not the territory: modern LLM work spans algorithms, systems, human feedback, products, and policy.

Broader context: Bommasani et al., "On the Opportunities and Risks of Foundation Models," 2021; Kaplan et al., "Scaling Laws for Neural Language Models," 2020; Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022; Anil et al., "PaLM 2 Technical Report," 2023; Nanda et al., "Progress Measures for Grokking via Mechanistic Interpretability," 2023.